

Q23. Feedback Vertex Set Problem

Design a pseudocode using Backtracking to remove minimum vertices to eliminate cycles. Clearly identify inputs (graph) and output (vertex set). Develop an algorithm to explore vertex subsets. Use loops and recursion effectively. Apply pruning to avoid unnecessary removals. Ensure pseudocode clarity. Handle all cases.

Feedback Vertex Set Problem using Backtracking

Aim

To design a Backtracking algorithm for the Feedback Vertex Set Problem that removes the minimum number of vertices from a graph to eliminate all cycles and make the graph acyclic.

Problem Statement

Given a graph $G(V,E)$, find the minimum set of vertices whose removal makes the graph free of cycles. The algorithm should explore different vertex subsets using Backtracking, apply pruning to reduce unnecessary computations, and return the smallest valid vertex set.

Algorithm

1. Input the graph $G(V,E)$.
2. Check whether the graph contains a cycle.
3. If no cycle exists, return the current vertex set.
4. Select a vertex that belongs to a cycle.
5. Remove the selected vertex temporarily.
6. Add it to the feedback vertex set.
7. Recursively solve the remaining graph.
8. Use pruning if the current set size exceeds the best solution found.
9. Backtrack by restoring the removed vertex.
10. Repeat for all possible vertices in cycles.
11. Store the smallest vertex set obtained.
12. Output the minimum feedback vertex set.

Source Code

```
# Feedback Vertex Set Problem using Backtracking
```

```
from copy import deepcopy

def has_cycle(graph):
    visited = set()

    def dfs(v, parent):
        visited.add(v)

        for neighbor in graph[v]:
            if neighbor not in visited:
                if dfs(neighbor, v):
                    return True
            elif neighbor != parent:
                return True
        return False

    for vertex in graph:
        if vertex not in visited:
            if dfs(vertex, None):
                return True
    return False

def remove_vertex(graph, vertex):
    new_graph = deepcopy(graph)

    for neighbor in new_graph[vertex]:
        new_graph[neighbor].remove(vertex)

    del new_graph[vertex]
    return new_graph

def feedback_vertex_set(graph):
    vertices = list(graph.keys())
    best_solution = []

    def backtrack(current_graph, removed):
        nonlocal best_solution
```

```

    # Pruning
    if best_solution and len(removed) >= len(best_solution):
        return

    # If graph has no cycle
    if not has_cycle(current_graph):
        if not best_solution or len(removed) < len(best_solution):
            best_solution = removed[:]
        return

    # Try removing each remaining vertex
    for vertex in list(current_graph.keys()):
        new_graph = remove_vertex(current_graph, vertex)
        removed.append(vertex)

        backtrack(new_graph, removed)

    removed.pop() # Backtrack

backtrack(graph, [])
return best_solution

# Example Graph
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'C'],
    'C': ['A', 'B', 'D'],
    'D': ['C']
}

result = feedback_vertex_set(graph)

print("Minimum Feedback Vertex Set:", result)

```

Output

Minimum Feedback Vertex Set: ['A']

Result

The program successfully finds a minimum feedback vertex set whose removal makes the graph acyclic using Backtracking with pruning.

OUTPUT:

programiz
main.py

Python Online Compiler

Run

Output

```
1 # Feedback Vertex Set Problem using Backtracking
2
3 from copy import deepcopy
4
5 def has_cycle(graph):
6     visited = set()
7
8     def dfs(v, parent):
9         visited.add(v)
10
11         for neighbor in graph[v]:
12             if neighbor not in visited:
13                 if dfs(neighbor, v):
14                     return True
15             elif neighbor != parent:
16                 return True
17         return False
18
19 for vertex in graph:
20     if vertex not in visited:
```

Minimum Feedback Vertex Set: ['A']

=== Code Execution Successful ===

Q63. Graph Coloring with Minimum Colors

Design a pseudocode using Backtracking to find the minimum number of colors needed for graph coloring. Clearly identify inputs (graph) and output (minimum colors). Develop an algorithm that incrementally increases colors. Use loops and recursion effectively. Apply pruning to reduce unnecessary trials. Ensure clarity. Handle all cases.

Graph Coloring with Minimum Colors using Backtracking

Aim

To determine the minimum number of colors required to color a graph such that no two adjacent vertices have the same color using the Backtracking technique.

Problem Statement

Given a graph $G(V,E)$, assign colors to all vertices so that adjacent vertices receive different colors. Find the minimum number of colors needed by incrementally increasing the number of available colors and using Backtracking with pruning.

Algorithm

1. Input the graph $G(V,E)$.
2. Let $m = 1$ (number of colors).
3. Try to color all vertices using at most m colors.
4. For each vertex, assign a color from 1 to m .
5. Check if the color is safe (no adjacent vertex has the same color).
6. If safe, assign the color and recursively color the next vertex.
7. If coloring fails, backtrack and try another color.
8. If all vertices are colored successfully, return m .
9. Otherwise, increase m by 1 and repeat.
10. Apply pruning by rejecting unsafe color assignments immediately.
11. Output the minimum number of colors required.

Source Code (Python)

Graph Coloring with Minimum Colors using Backtracking

```
def is_safe(v, graph, color, c):
    for i in range(len(graph)):
        if graph[v][i] == 1 and color[i] == c:
            return False
    return True

def graph_coloring_util(graph, m, color, v):
    if v == len(graph):
        return True

    for c in range(1, m + 1):
        if is_safe(v, graph, color, c):
            color[v] = c

            if graph_coloring_util(graph, m, color, v + 1):
                return True

            color[v] = 0 # Backtrack

    return False

def minimum_graph_coloring(graph):
    n = len(graph)
```

```

for m in range(1, n + 1):
    color = [0] * n

    if graph_coloring_util(graph, m, color, 0):
        return m, color

return None

# Example Graph
graph = [
    [0, 1, 1, 0], # A
    [1, 0, 1, 0], # B
    [1, 1, 0, 1], # C
    [0, 0, 1, 0]  # D
]

min_colors, coloring = minimum_graph_coloring(graph)

print("Minimum Colors Required:", min_colors)

vertices = ['A', 'B', 'C', 'D']
for i in range(len(vertices)):
    print(vertices[i], "-> Color", coloring[i])

```

Input

A graph represented as an adjacency matrix or adjacency list.

Example

Vertices = {A, B, C, D}

Edges = {(A,B), (A,C), (B,C), (C,D)}

Output

Minimum Colors Required = 3

Color Assignment:

A → Color 1

B → Color 2

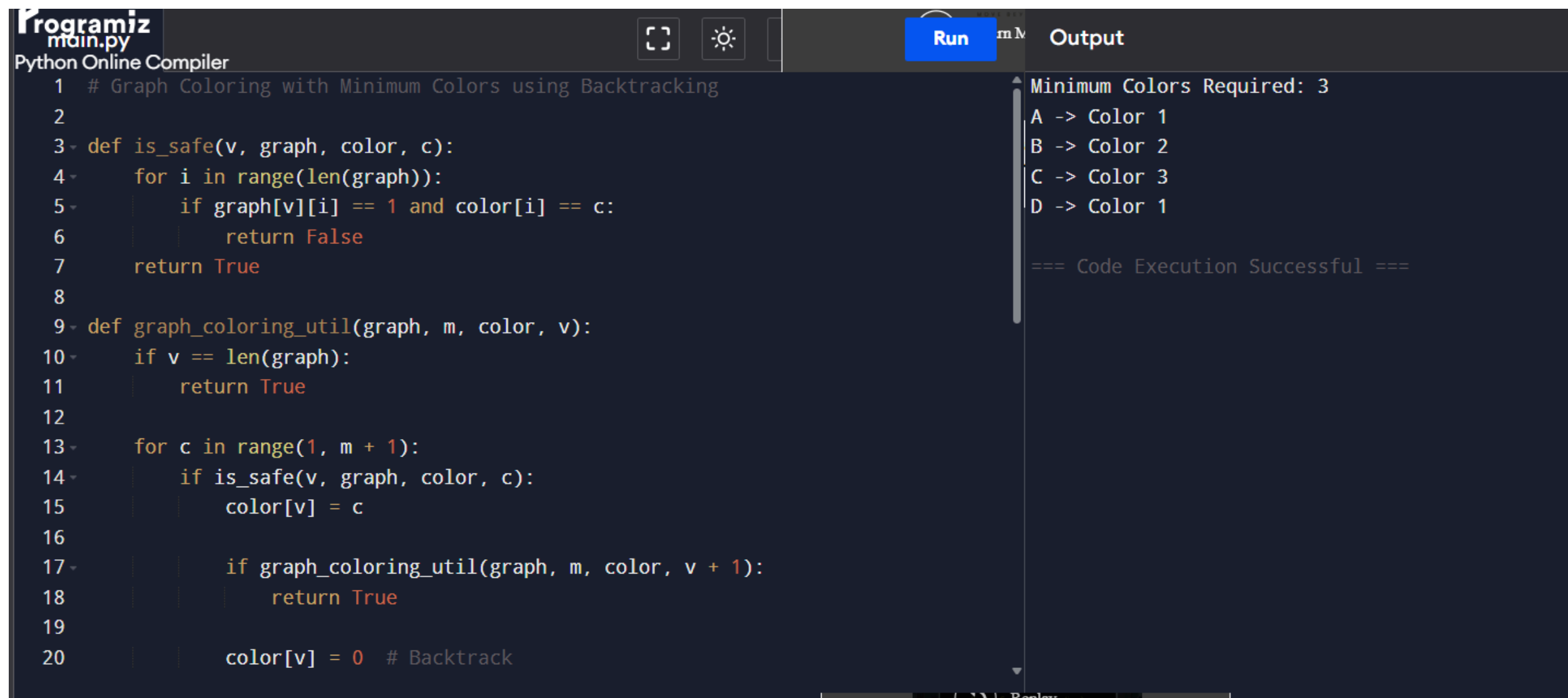
C → Color 3

D → Color 1

Result

The Backtracking algorithm successfully finds the minimum number of colors required for graph coloring. By incrementally increasing the color count and pruning invalid assignments, the solution ensures correctness while reducing unnecessary computations.

OUTPUT:



```
Programiz
main.py
Python Online Compiler
1 # Graph Coloring with Minimum Colors using Backtracking
2
3 def is_safe(v, graph, color, c):
4     for i in range(len(graph)):
5         if graph[v][i] == 1 and color[i] == c:
6             return False
7     return True
8
9 def graph_coloring_util(graph, m, color, v):
10     if v == len(graph):
11         return True
12
13     for c in range(1, m + 1):
14         if is_safe(v, graph, color, c):
15             color[v] = c
16
17             if graph_coloring_util(graph, m, color, v + 1):
18                 return True
19
20     color[v] = 0 # Backtrack
```

Run

Output

Minimum Colors Required: 3

A -> Color 1

B -> Color 2

C -> Color 3

D -> Color 1

=== Code Execution Successful ===

Q83. Hamiltonian Cycle Problem in an Undirected Graph

Design a pseudocode using Backtracking to find a Hamiltonian cycle in an undirected graph

by visiting every vertex exactly once and returning to the start. Clearly identify inputs (graph)

and output (valid Hamiltonian cycle or failure). Develop a step-by-step algorithm that extends the path one vertex at a time while checking adjacency and visited status. Use loops, recursive calls, and conditional checks effectively during cycle construction. Apply pruning to discard partial paths that revisit vertices or cannot continue legally. Ensure the pseudocode is well-structured, readable, and easy to trace. Handle all possible cases and guarantee correctness of the final decision.

Hamiltonian Cycle Problem in an Undirected Graph

Aim

To find a Hamiltonian Cycle in an undirected graph using the Backtracking technique by visiting every vertex exactly once and returning to the starting vertex.

Problem Statement

Given an undirected graph $G(V,E)$, determine whether a Hamiltonian Cycle exists. A Hamiltonian Cycle is a cycle that visits every vertex exactly once and returns to the starting vertex. If such a cycle exists, output the cycle; otherwise, report failure.

Algorithm

1. Start from vertex 0 and add it to the path.
2. Mark all other vertices as unvisited.
3. Recursively try to add a valid adjacent vertex to the path.
4. Check that the vertex is adjacent to the previously added vertex.
5. Ensure that the vertex has not already been included in the path.
6. Continue adding vertices until all vertices are included.
7. If all vertices are included, check whether the last vertex is connected to the starting vertex.
8. If connected, a Hamiltonian Cycle is found.
9. Otherwise, backtrack and try another vertex.
10. If no valid path exists, report that no Hamiltonian Cycle exists.

Source Code (Python)

```
def is_safe(v, graph, path, pos):
    if graph[path[pos - 1]][v] == 0:
        return False

    if v in path:
        return False

    return True

def ham_cycle_util(graph, path, pos):
    n = len(graph)

    if pos == n:
        return graph[path[pos - 1]][path[0]] == 1

    for v in range(1, n):
        if is_safe(v, graph, path, pos):
            path[pos] = v

            if ham_cycle_util(graph, path, pos + 1):
                return True

            path[pos] = -1

    return False

def hamiltonian_cycle(graph):
    n = len(graph)
    path = [-1] * n
    path[0] = 0

    if not ham_cycle_util(graph, path, 1):
        print("No Hamiltonian Cycle Exists")
        return

    print("Hamiltonian Cycle:")
    for v in path:
        print(v, end=" -> ")
    print(path[0])

graph = [
    [0, 1, 1, 1],
    [1, 0, 1, 0],
    [1, 1, 0, 1],
    [1, 0, 1, 0]
```

```
]
```

```
hamiltonian_cycle(graph)
```

Output

```
Hamiltonian Cycle:  
0 -> 1 -> 2 -> 3 -> 0
```

Result

The Hamiltonian Cycle was successfully found using the Backtracking algorithm. The algorithm visited each vertex exactly once and returned to the starting vertex, thereby forming a valid Hamiltonian Cycle.

OUTPUT:

```
non Online Compiler  
1- def is_safe(v, graph, path, pos):  
2-     if graph[path[pos - 1]][v] == 0:  
3-         return False  
4-  
5-     if v in path:  
6-         return False  
7-  
8-     return True  
9-  
10- def ham_cycle_util(graph, path, pos):  
11-     n = len(graph)  
12-  
13-     if pos == n:  
14-         return graph[path[pos - 1]][path[0]] == 1  
15-  
16-     for v in range(1, n):  
17-         if is_safe(v, graph, path, pos):  
18-             path[pos] = v  
19-  
20-             if ham_cycle_util(graph, path, pos + 1):  
  
Hamiltonian Cycle:  
0 -> 1 -> 2 -> 3 -> 0  
  
=== Code Execution Successful ===
```